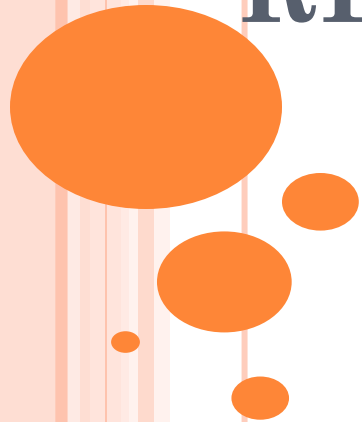


PROGRAMACIÓN REACTIVA CON RX JAVA



SOBRE RX JAVA

- **Librería Java para la creación de aplicaciones según el paradigma reactivo**
- **Trabaja con flujos de datos asíncronos**
- **Manejo de eventos no bloqueantes**
- **Utilizada fundamentalmente para la creación de aplicaciones Java estándar reactivas**



AYUDA OFICIAL Y DEPENDENCIAS

➤ **Para poder utilizar Rx Java debemos incluir la siguiente dependencia en el proyecto:**

```
<dependency>  
  <groupId>io.reactivex.rxjava3</groupId>  
  <artifactId>rxjava</artifactId>  
  <version>3.1.8</version>  
</dependency>
```

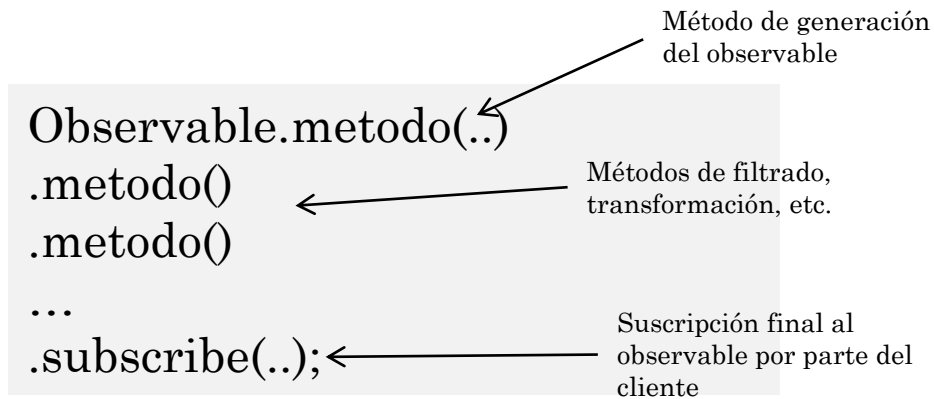
➤ **Se puede encontrar la ayuda oficial sobre las clases e interfaces en la siguiente dirección:**

<https://javadoc.io/doc/io.reactivex.rxjava3/rxjava/latest/index.html>



LA CLASE OBSERVABLE

- Es la clase más importante del API RxJava.
- Proporciona una serie de métodos para generar flujos de datos desde el emisor, así como métodos para que el cliente pueda suscribirse a estos flujos de datos.
- Estilo de programación funcional:



EJEMPLO BÁSICO

➤ Método que genera un flujo de cadenas de caracteres:

```
public class NombresService {  
    public Observable<String> getNombres(){  
        return Observable.just("uno","dos","tres", "cuatro");  
    }  
}
```

➤ Cliente que se suscribe al flujo:

```
public class Test {  
    public static void main(String[] args) throws InterruptedException {  
        var service=new NombresService();  
        service.getNombres()  
        .subscribe(System.out::println,  
            e->System.out.println(e.getMessage()),  
            ()->System.out.println("Termino"));  
  
        Thread.sleep(3000);  
    }  
}
```

consumer con la tarea a
realizar cuando llegue cada
elemento del flujo

tarea en caso de
error

espera un rato hasta que finalice
la suscripción

acción al finalizar el
flujo



CREACIÓN DE UN OBSERVABLE

- Un observable puede crearse a partir de diferentes fuentes de datos.
- Para ello, la propia clase Observable proporciona métodos estáticos. Algunos de los más importantes:
 - `just(T i1, T i2,...)`. Crea un observable a partir de una serie discreta de datos (hasta 10)
 - `fromArray(T...items)`. Crea un observable a partir de un array o un número variable de argumentos
 - `fromIterable(Iterable<T> iter)`. Crea un array a partir de un Iterable, como una lista o conjunto



CREACIÓN DE UN OBSERVABLE II

- **fromStream(Stream<T> st).** Crea un observable a partir de un Stream
- **fromFuture(Future<T> fut).** Convierte un Future en un Observable
- **interval(long period, TimeUnit unit).** Genera un observable de Long, que emite una secuencia infinita de números naturales comenzando por 0L. El parámetro *period* indica el retardo entre cada número.



SUSCRIPCIÓN A UN OBSERVABLE

➤ Para suscribirse a un observable, disponemos de los siguientes métodos:

- `subscribe(Consumer<T> consumer)`. A través del consumer establece la tarea a realizar cuando se reciba un dato de respuesta. La llamada a *subscribe()* es asíncrona, no bloquea el hilo actual.
- `subscribe(Consumer<T> consumer, Consumer<Throwable> error, Action final)`. Igual que el anterior, pero con una tarea en caso de error y al finalizar el flujo de datos
- `blockingSubscribe(..)`. Bloquea el hilo actual hasta que se complete el flujo o se produzca un error



MANIPULACIÓN DE FLUJOS

➤ Una vez creado un observable, la clase dispone de una serie de métodos para manipular el flujo de datos:

- `filter(Predicate<T> consumer)`. Filtra los datos emitidos por el observable, devolviendo uno nuevo con los que cumplen la condición
- `map(Function<T,R> fun)`. Transforma los elementos emitidos por el observable y devuelve uno nuevo con los resultados de dichas transformaciones
- `flatMap(Function<T,ObservableSource<R>> fun)`. Transforma cada elemento en un `Observable<R>` y finalmente los combina en un único observable
- `concatMap(Function<T,ObservableSource<R>> fun)`. Igual que el anterior pero respetando el orden



MANIPULACIÓN DE FLUJOS II

- **take(long n).** Devuelve un nuevo observable con los n primeros elementos.
- **skip(long n).** Devuelve un nuevo observable saltandose los n primeros elementos.
- **sorted(Comparator<T> comp).** Devuelve un observable resultante de ordenar los elementos del observable principal.
- **distinct().** Devuelve un nuevo Observable sin duplicados.
- **firstElement()/lastElement().** Devuelve el primer/último elemento del flujo respectivamente. El tipo de devolución es **Maybe<T>**, que también proporciona un método **subscribe()** para que el cliente se suscriba al valor emitido



FLATMAP, CONCATMAP Y SWITCHMAP

➤ Realizan función similar, con diferencias de funcionamiento:

- **flatMap**. Lanza las funciones en modo concurrente, por lo que el orden de los observables obtenidos, y por tanto del resultado final, no es predecible
- **concatMap**. Funciona en modo secuencial. El orden final de los elementos está garantizado
- **switchMap**. Cuando llega un nuevo elemento de la fuente, cancela la emisión anterior, por lo que se queda solo con el observable emitido por el último elemento:

```
Observable<String> fuenteRapida = Observable.just("A", "B", "C", "D", "E", "F");  
fuenteRapida.switchMap(s->Observable.just(s).delay(500,MILLISECONDS))  
.subscribe(v->System.out.println("SwitchMap emite: " + v));
```

Solo muestra F

MANEJO DE ERRORES

➤ Los errores se pueden manejar de dos formas:

- Métodos `onErrorReturn`, `onErrorComplete`,... de `Observable`. El error se controla en el flujo y este no se interrumpe. Se puede transformar el error en algo para que el flujo continúe

```
Observable.just("1", "2", "x", "4")
    .map(Integer::parseInt)
    .onErrorReturn(err -> -1) // convierte el error en un valor
    .subscribe(System.out::println);
```

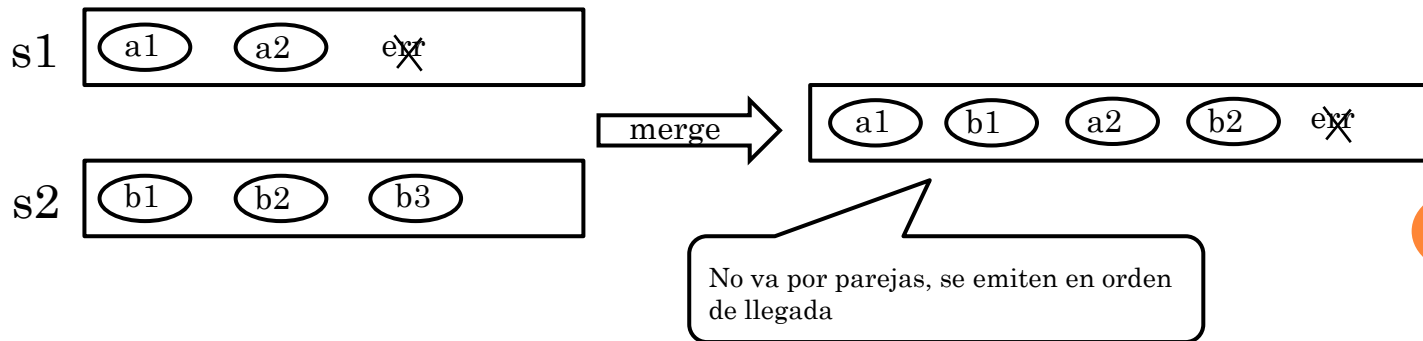
- Segundo parámetro de `subscribe`. Si se produce un error en el flujo sin capturar, este se interrumpe y se pasa el control del error al cliente, que lo capturará en `subscribe`

```
Observable.just("1", "2", "x", "4")
    .map(Integer::parseInt)
    .subscribe(
        System.out::println,
        err -> System.err.println("Error capturado: " + err)
    );
```



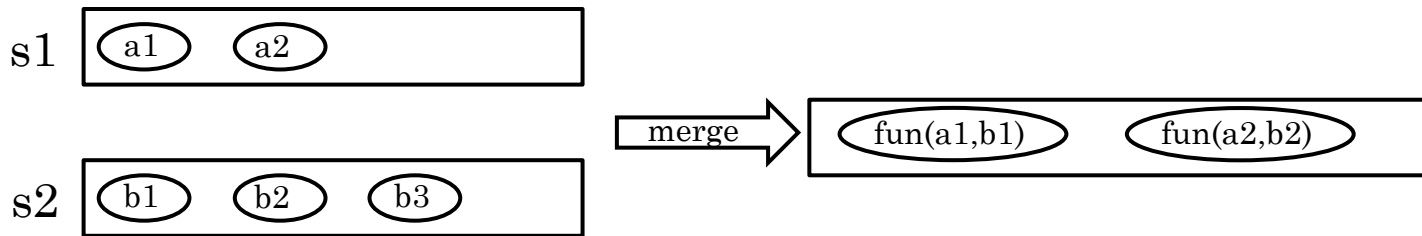
COMBINAR OBSERVABLES

- En muchas ocasiones resulta útil combinar dos o más observables en un único observable.
- Los siguientes métodos permiten la combinación de dos observables en uno:
 - `merge(ObservableSource<T> s1, ObservableSource<T> s2 )`. Devuelve un nuevo observable que es la combinación de los dos observables parámetro. Si un observable emite error, finaliza la combinación:



COMBINAR OBSERVABLES II

▪ `zip(ObservableSource<T1> s1, ObservableSource<T2> s2 , BiFunction<T1,T2,R> fun)`. Igual que `merge()`, solo que aplica una función de transformación sobre cada pareja de elementos, devolviendo un observable resultado de dicha transformación. Las fuentes a combinar no tienen que ser del mismo tipo:



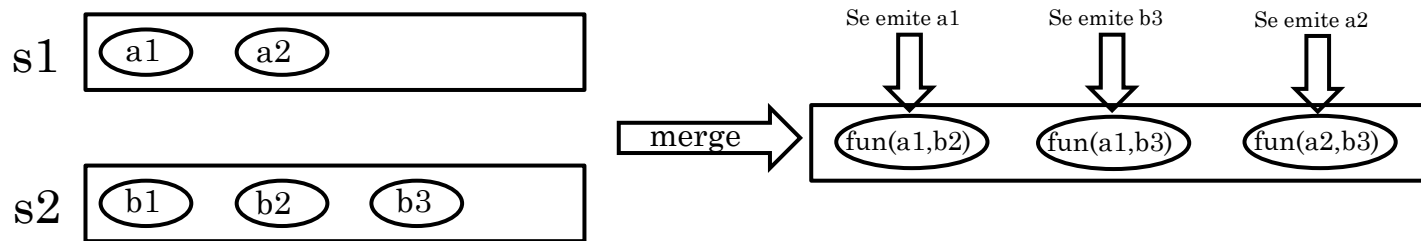
La combinación finaliza al completar el elemento más corto

Si alguna de las fuentes emite error, se cancela la emisión



COMBINAR OBSERVABLES III

▪ `combineLatest(ObservableSource<T1> s1, ObservableSource<T2> s2, BiFunction<T1,T2,R> fun)`. Similar a `zip`, pero en lugar de combinar cada pareja de nuevos valores, `combineLatest` combina cada nuevo elemento emitido por alguna de las fuentes con el último disponible de la otra. Por ejemplo, si el orden de emisión es `b1, b2, a1, b3, a2`, el resultado es :



La combinación no comienza hasta que haya un valor de cada fuente

El primer dato de `s2` (`b1`) se ha perdido

EJEMPLO GPS

- El siguiente ejemplo corresponde a un simulador de GPS que emite la velocidad del coche y tiempo estimado de llegada.
- Usa *combineLatest* para disponer siempre de la última combinación

```
// Fuente 1: velocidad del coche (cada 2s)
Observable<Integer> velocidad = Observable.interval(2, TimeUnit.SECONDS)
    .map(n -> 40 + (int)(Math.random() * 60)); // entre 40 y 100 km/h

// Fuente 2: tiempo estimado de llegada (cada 5s)
Observable<Integer> tiempoEstimado = Observable.interval(5, TimeUnit.SECONDS)
    .map(n -> 30 + (int)(Math.random() * 20)); // entre 30 y 50 min

// Combinamos las dos fuentes
Observable.combineLatest(velocidad, tiempoEstimado,
    (v, t) -> "Velocidad: " + v + " km/h | Tiempo estimado: " + t + " min")
    .subscribe(System.out::println);
```

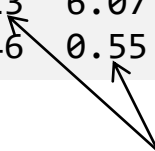


MULTICAST. OBSERVABLE FRÍO

➤ Si varios suscriptores se suscriben a un observable, cada uno obtendrá su propia copia y, por tanto, su propio flujo de datos. Es lo que se llama un Observable frío:

```
var obsCold=Observable.interval(3, TimeUnit.SECONDS)
    .map(n->Math.random()*10);
obsCold.subscribe(System.out::println); //1.23  6.07  4.8 ...
obsCold.subscribe(System.out::println); //4.46  0.55  9.11 ...
```

Cada suscripción genera un
flujo diferente



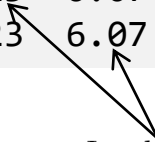
MULTICAST. OBSERVABLE CALIENTE

➤ Un observable caliente es aquel que genera un flujo único, por lo que todas las llamadas a subscribe trabajarán con los mismos datos.

➤ Para convertir un observable en caliente llamamos a `share()`:

```
var obsCold=Observable.interval(3, TimeUnit.SECONDS)
    .map(n->Math.random()*10).share();
obsCold.subscribe(System.out::println); //1.23 6.07 4.8 ...
obsCold.subscribe(System.out::println); //1.23 6.07 4.8 ...
```

Los dos suscriptores ven los mismos datos



➤ Un observable caliente permite realizar un multicast, que no es otra cosa que poder transmitir los mismos datos a varios suscriptores

EL PROBLEMA DEL EXCESO DE DATOS

➤ Cuando un productor emite datos a una velocidad mucho mayor de la que el cliente los puede procesar, el cliente puede presentar problemas:

- Consumo excesivo de memoria. Los datos que llegan y no se procesan se acumulan en memoria y pueden provocar `OutOfMemoryError`
- Datos obsoletos. Si la diferencia de velocidad es grande, cuando el cliente procesa un dato este puede estar desfasado (no apto en aplicaciones de bolsa, GPS, etc.)
- Inestabilidad. El hilo consumidor se puede saturar y volverse impredecible.



CONTRAPRESIÓN (BACKPRESSURE)

- **Consiste en aplicar medidas para paliar la descompensación entre las velocidades de emisión y procesado.**
- **El productor debe utilizar Flowable en lugar de Observable**
- **Mediante los siguientes métodos de Flowable se indica cual será el comportamiento si el consumidor se satura de datos:**
 - **onBackpressureBuffer().** Almacena en un buffer los que el cliente no puede procesar
 - **onBackpressureDrop().** Descarta los valores que no puede procesar el cliente
 - **onBackpressureLatest().** Descarta todos menos el último
- **Los efectos se aprecian si están en distintos hilos**



EJEMPLO CONTRAPRESIÓN CON BUFFER

Productor

```
public class GeneraDatos {  
    public Flowable<Long> getDataContrapresion(){  
        return Flowable.interval(10, TimeUnit.MILLISECONDS)  
            .take(500); //genera números del 0 al 499  
            .onBackpressureBuffer(400); //buffer de 400 elementos  
    }  
}
```

Consumidor

```
var generaDatos=new GeneraDatos();  
generaDatos.getDataContrapresion()  
    .observeOn(Schedulers.io()) // consumidor en otro hilo  
    .subscribe(d->{  
        try {  
            Thread.sleep(100); //simula retardo  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.println("Recibido: "+d);  
    });
```

Salida

```
Recibido: 0  
Recibido: 1  
:  
Recibido: 499
```

Ningún dato se pierde, el consumidor lo recibe del buffer. Aunque depende del tamaño del buffer y de la velocidad del cliente. Si se llena, excepción



EJEMPLO CONTRAPRESIÓN CON DROP

Productor

```
public class GeneraDatos {  
    public Flowable<Long> getDataContrapresion(){  
        return Flowable.interval(10, TimeUnit.MILLISECONDS)  
            .take(500); //genera números del 0 al 499  
            .onBackpressureDrop(); //los que no puede emitir se descartan  
    }  
}
```

Consumidor

```
var generaDatos=new GeneraDatos();  
generaDatos.getDataContrapresion()  
    .observeOn(Schedulers.io()) // consumidor en otro hilo  
    .subscribe(d->{  
        try {  
            Thread.sleep(100); //simula retardo  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.println("Recibido: "+d);  
    });
```

Salida

```
Recibido: 0  
Recibido: 1  
Recibido: 2  
:  
Recibido: 127
```

El tamaño de la memoria local
del cliente solo permite hasta
127, los restantes no llegan



EJEMPLO CONTRAPRESIÓN CON LATEST

Productor

```
public class GeneraDatos {  
    public Flowable<Long> getDataContrapresion(){  
        return Flowable.interval(10, TimeUnit.MILLISECONDS)  
            .take(500); //genera números del 0 al 499  
            .onBackpressureLatest(); //los que no puede emitir se  
            descartan, menos el último  
    }  
}
```

Consumidor

```
var generaDatos=new GeneraDatos();  
generaDatos.getDataContrapresion()  
    .observeOn(Schedulers.io()) // consumidor en otro hilo  
    .subscribe(d->{  
        try {  
            Thread.sleep(100); //simula retardo  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.println("Recibido: "+d);  
    });
```

Salida

```
Recibido: 0  
Recibido: 1  
Recibido: 2  
:  
Recibido: 127  
Recibido: 499
```

El último dato del bloque
se envía y es procesado
por el cliente



CONCURRENCIA CON SCHEDULERS

- Un Scheduler determina el tipo de hilo con el que se va a ejecutar un proceso en un productor o consumidor.
- Existen los siguientes tipos de hilos:
 - Scheduler.single(). Tipo de hilo por defecto, hace que todo se ejecute en un único hilo
 - Scheduler.io(). Utiliza un pool de hilos de tamaño variable, útil para operaciones de entrada/salida
 - Scheduler.computation(). Utiliza un pool con tantos hilos como núcleos de CPU. Adecuado para cálculos costosos



APLICACIÓN DE UN SCHEDULER

➤ Un Scheduler se puede aplicar sobre el flujo del productor o del consumidor.

➤ Para aplicarlo sobre el flujo del productor utilizamos el método de Observable:

▪ **subscribeOn(Scheduler sc).** Aplica el Scheduler sobre el flujo del productor:

```
Observable.range(1, 3)
    .subscribeOn(Schedulers.io())    // productor en IO
    .map(i -> {
        System.out.println("map1: " + i + " - " + Thread.currentThread().getName());
        return i;
    })
    .subscribe(i ->
        System.out.println("subscribe: ")
    );
System.in.read();
```



```
map1: 1 - RxCachedThreadScheduler-1
subscribe:
map1: 2 - RxCachedThreadScheduler-1
subscribe:
map1: 3 - RxCachedThreadScheduler-1
subscribe:
```

▪ Si se llama varias veces a subscribeOn() solo tendrá efecto la primera llamada

▪ interval() siempre usa computation. No le afecta subscribeOn()

APLICACIÓN DE UN SCHEDULER

➤ Para aplicarlo sobre el flujo del consumidor utilizamos el método de Observable:

▪ **observeOn(Scheduler sc).** Aplica el Scheduler sobre el flujo del consumidor:

```
Observable.range(1, 3)
    .observeOn(Schedulers.io())    // productor en IO
    .map(i -> {
        System.out.println("map1: " + i + " - " + Thread.currentThread().getName());
        return i;
    })
    .observeOn(Schedulers.computation()) // a partir de aquí, en computation
    .map(i -> {
        System.out.println("map2: " + i + " - " + Thread.currentThread().getName());
        return i;
    })
    .subscribe(i -> {
        System.out.println("subscribe: " + i + " - " + Thread.currentThread().getName())
    })
```



```
map1: 1 - RxCachedThreadScheduler-1
map1: 2 - RxCachedThreadScheduler-1
map1: 3 - RxCachedThreadScheduler-1
map2: 1 - RxComputationThreadPool-1
subscribe: 1 - RxComputationThreadPool-1
map2: 2 - RxComputationThreadPool-1
subscribe: 2 - RxComputationThreadPool-1
map2: 3 - RxComputationThreadPool-1
subscribe: 3 - RxComputationThreadPool-1
```

▪ Se puede llamar varias veces a observeOn() para cambiar de scheduler y que posteriores procesamientos ocurran en otro hilo. Si no hay observeOn(), el consumidor se ejecuta en el mismo hilo que el productor.